How to train the model

- step 1:- we read the data

- step 2:- we deivide the data into X and Y

- step 3:- we did train_test_split

- step 4:- we train the data or model

In [46]:
```
# how to seperate  the data [ IMP ]

from sklearn import datasets
from sklearn.model_selection import train_test_split
#   <module>     <class>               <method>
from sklearn.linear_model import LinearRegression


california_df = datasets.fetch_california_housing(as_frame=True).frame

x = california_df.drop("MedHouseVal",axis = 1)  # this is a input columns
y = california_df['MedHouseVal']              # this is output or prediction column

x_train, x_test, y_train, y_test = train_test_split(x,y,
                                                    test_size=0.2,
                                                    random_state = 42)  #--> this d

lr = LinearRegression()
lr.fit(x_train, y_train)        # it is model { our model name is lr }

# y_test_pred_all = lr.intercept_ + np.sum(lr.coef_ * x_test.values,axis=1)     # c
# y_train_pred_all = lr.intercept_ + np.sum(lr.coef_ * x_train.values,axis=1)   # c
```

Out[46]:

▾ LinearRegression

▸ Parameters

In [48]:
```
import numpy as np

#                       x1              x2              x3                  x4
#y_pred = b0 + b1 * 'MedInc' + b2 * 'HouseAge' + b3 * 'AveRooms' + b4 * 'AveBedrms'

y_test_pred_all = lr.intercept_ + np.sum(lr.coef_ * x_test.values,axis=1)     # com
y_train_pred_all = lr.intercept_ + np.sum(lr.coef_ * x_train.values,axis=1)   # com
```

```
# comparision y_train and y_pred

df1_train = x_train.copy()
df1_train['y_train'] = y_train
df1_train['y_train_pred_all'] = y_train_pred_all
df1_train['error'] = y_train - y_train_pred_all   # calculate error
# df1_train['squared error'] = np.square(y_train - y_train_pred_all )
#############################################################################
# comparision y_train and y_pred
# df2_test = x_test.copy()                  # x_test means a actual point
# df2_test['y_test']= y_test
# df2_test['y_test_pred_all'] = y_test_pred_all   # and it is a prediction value
# df2_test['error'] = y_test - y_test_pred_all  # actual - predict to cal error
# df2_test['squared error'] = np.square(y_test - y_test_pred_all)
```

In [ ]:
```
# we have each square error
# SSE ,MSE ,RMSE--- TRAIN
```

In [ ]:

In [ ]:

In [ ]:

In [11]:
```
# how many inputs we have = 8  [ x ] --->  { x is input columns }
x.columns
# # how many output columns we have = 1 [ y ] ---> and { y is predition or output c
y.columns
```

In [7]:
```
# Explainations:-
# linear regression formula:-
#                       x1              x2              x3                  x4
y_pred = b0 + b1 * 'MedInc' + b2 * 'HouseAge' + b3 * 'AveRooms' + b4 * 'AveBedrms'
# y_pred_0 = lr.intercept_ + np.sum(lr.coef_ * x_test.values[0])
# b0 = intercept and b1 = coefficient or slope
```

In [ ]:

In [15]:
```
print(lr.coef_)                 # coefficient = slope { means how much Y increase w
lr.intercept_             # means { frome where start prediction line in Y }
```

```
[ 4.48674910e-01  9.72425752e-03 -1.23323343e-01  7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
```

Out[15]:
```
np.float64(-37.023277706064235)
```

In [16]:
```
# we have x_test values              we compare the x_test value with a y_test valu
x_test.values[0]
```

Out[16]:
```
array([ 1.68120000e+00,  2.50000000e+01,  4.19220056e+00,  1.02228412e+00,
        1.39200000e+03,  3.87743733e+00,  3.60600000e+01, -1.19010000e+02])
```

In [ ]:

- multiple linear regression code step by step with explaination [chatgpt]

In [ ]:

In [1]:
```
# stp1 :- import important librabry

from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

import numpy as np
```

In [2]:
```
# stp2:- load california housing datasets [ load the data ]

cal_df = datasets.fetch_california_housing(as_frame= True).frame
# cal_df.columns
```

In [3]:
```
#stp3:- seperate the input(x) and output (y) data

X = cal_df.drop("MedHouseVal",axis = 1)
y = cal_df["MedHouseVal"]
```

In [4]:
```
# stp4:- Split the data into train and test

X_train, X_test, y_train, y_test = train_test_split( X,y,
                                                     test_size = 0.2,
                                                     random_state = 42 )
```

In [5]:
```
# step 5 :- create a liner regression_model

lr = LinearRegression()
```

In [6]:
```
# stp6 :- Train the model

lr.fit(X_train, y_train)          # note the bo and b1 is here [ bo = lr.intercept
```

Out[6]:
```
▾ LinearRegression i ?

▸ Parameters
```

In [7]:
```
# get a intercept [b0] & coefficent [b1]
lr.intercept_
lr.coef_
```

Out[7]:
```
array([ 4.48674910e-01,  9.72425752e-03, -1.23323343e-01,  7.83144907e-01,
       -2.02962058e-06, -3.52631849e-03, -4.19792487e-01, -4.33708065e-01])
```

In [35]:
```
# stp7 formula :- predict Train and Test data
'''                    x1              x2              x3                  x4
```

```
# y_pred = b0 + b1 * 'MedInc' + b2 * 'HouseAge' + b3 * 'AveRooms' + b4 * 'AveBedrms

# b0 = intercept and b1 = coefficient or slope
```

In [8]:
```
# stp7 :- predict Train and Test data

y_test_pred_all = lr.intercept_ + np.sum(lr.coef_ * X_test.values,axis=1)     # F:-
y_train_pred_all = lr.intercept_ + np.sum(lr.coef_ * X_train.values,axis = 1)
```

In [15]:
```
# stp 8:- Get all Train SSE MSE & RMSE data

df1_train = X_train.copy()

df1_train['y_train'] = y_train                    # ------------------------------
df1_train['y_train_pred_all'] = y_train_pred_all  #------------- Predict value

# err = Actual values - Pred calues
df1_train['error'] = y_train - y_train_pred_all   #-------> errr calucation
df1_train['squared error'] = np.square(y_train - y_train_pred_all) #-----> squared
#----------------------------
train_SSE = np.sum(np.square(y_train - y_train_pred_all))
train_MSE = np.sum(np.square(y_train - y_train_pred_all))/len(y_train)
train_RMSE = np.sqrt(train_MSE)
```

In [26]:
```
# stp 9:- Get all Test SSE MSE & RMSE data

df2_test = X_test.copy()

df2_test['y_test'] = y_test                    # ------------------------------ Ac
df2_test['y_test_pred_all'] = y_test_pred_all  #------------- Predict value

# err = Actual values - Pred calues
df2_test['error'] = y_test - y_test_pred_all   #-------> errr calucation
df2_test['squared error'] = np.square(y_test - y_test_pred_all) # square of error
#-------------------------------------------------------------------------------
test_SSE = np.sum(np.square(y_test - y_test_pred_all))
test_MSE = np.sum(np.square(y_test - y_test_pred_all))/len(y_test)
test_RMSE = np.sqrt(test_MSE)
```

In [30]:
```
# stp 10 :- show the result
display(df1_train.head())
df2_test.head()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitu
14196	3.2596	33.0	5.017657	1.006421	2300.0	3.691814	32.71	-117
8267	3.8125	49.0	4.473545	1.041005	1314.0	1.738095	33.77	-118
17445	4.1563	4.0	5.645833	0.985119	915.0	2.723214	34.66	-120
14265	1.9425	36.0	4.002817	1.033803	1418.0	3.994366	32.69	-117
2271	3.5542	43.0	6.268421	1.134211	874.0	2.300000	36.78	-119

Out[30]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Long
20046	1.6812	25.0	4.192201	1.022284	1392.0	3.877437	36.06	-1
3024	2.5313	30.0	5.039384	1.193493	1565.0	2.679795	35.14	-1
15663	3.4801	52.0	3.977155	1.185877	1310.0	1.360332	37.80	-1
20484	5.7376	17.0	6.163636	1.020202	1705.0	3.444444	34.28	-1
9814	3.7250	34.0	5.492991	1.028037	1063.0	2.483645	36.62	-1

In [31]:
```
train_RMSE, test_RMSE
```

Out[31]:
```
(np.float64(0.7196757085831574), np.float64(0.745581383012776))
```

In [ ]:
```
'''complete steps:-'''
# stp1 :- import important librabry
# stp2:- load california housing datasets [ load the data ]---1 library
#stp3:- seperate the input(x) and output (y) data
# stp4:- Split the data into train and test ---- 2 library
# step 5 :- create a liner regression_model -----3 ib
# stp6 :- Train the model
# stp7 :- predict Train and Test data
# stp 8:- Train data comparision  === {err = Actual values - Pred calues}  ---4 lib
# stp 9:- Test data comparision === {err = Actual values - Pred calues}
# stp 10 :- show the result
```

In [ ]:

In [34]:

In [35]:

In [ ]: